PHASE

17

The RISC-V Port

Boot the same kernel and userland on a second architecture: QEMU's RISC-V virt board, with OpenSBI as the firmware and Turk-OS in supervisor mode

Stage V progress toward Turk-OS 2.0 (this phase in red) 20 % → 60 %

Duration	5–6 weeks (Task 17.13 on your own core is open-ended)
Difficulty	★★★★☆ (4/5)
You need	Phase 16 complete: the portable core, both cross compilers, the FDT parser and its host tests
You will have	Turk-OS on `qemu-system-riscv32 -machine virt`, booting to `tsh`: traps with a crash report that names every cause, the SBI timer at 100 Hz, the PLIC with UART input, an Sv32 higher-half kernel at `0xC0000000`, user mode through `ecall`, a virtio-blk disk carrying TurkFS, and `make ARCH=riscv32 test`. Optionally, a checked plan for running it on Timur-RV32IMC.

1 Why this phase

Phase 16 claimed that everything outside `kernel/arch/i386` is generic. This phase tests the claim on a second machine: you write the RISC-V half of `include/arch.h`, and the scheduler, the VFS, TurkFS, `fork` and the whole userland are recompiled, not rewritten. Every place where that fails is a leak the audit of Task 16.2 missed; fix it in the generic code, note it in `docs/porting.md`, and keep `make ARCH=i386 test` passing.

RISC-V comes first because it changes little at once. RV32 keeps the 32-bit word, and Sv32 is two-level 4 KiB paging like x86's, so the kernel stays at `0xC0000000` and Phase 10's user layout stays as it is. What changes is the boot path (firmware instead of GRUB), traps (CSRs instead of an IDT and a TSS), devices (a devicetree instead of fixed PC addresses) and the assembler (GNU `as` instead of NASM). And the road leads on to your own core (Task 17.13).

2 Concepts you need

2.1 RISC-V in one page

RISC-V is a load/store architecture: only loads and stores touch memory (`lw a0, 4(a1)` is x86's `mov eax, [ebx+4]`), and there is no flags register, since branches compare two registers (`blt a0, a1, label`). The registers `x0`–`x31` go by ABI names: `zero` (always 0), `ra` (return address), `sp`, `gp` and `tp` (unused by the kernel), `t0`–`t6` (temporaries), `s0`–`s11` (callee-saved) and `a0`–`a7` (arguments; `a0`, `a1` also results). `call` leaves the return address in `ra` instead of pushing it, and `sp` must be 16-byte aligned at every call.

The ISA string `rv32imac_zicsr_zifencei` means: 32-bit base integer (I), multiply and divide (M), atomics (A), compressed 16-bit encodings (C), the CSR instructions (Zicsr) and `fence.i` (Zifencei). With C, assembler and compiler choose 2-byte forms on their own, so an instruction is 2 or 4 bytes long; a 4-byte one has binary `11` in its lowest two bits. `-mabi=ilp32` means 32-bit `int`, `long` and pointers and no floating-point registers.

2.2 Three privilege modes and the SBI

A RISC-V hart (hardware thread) runs in machine (M), supervisor (S) or user (U) mode. Turk-OS runs in S-mode, programs in U-mode. Unlike the BIOS and GRUB, the firmware, **OpenSBI**, stays resident in M-mode and offers services through the **Supervisor Binary Interface** (SBI), called with `ecall` one level down (Figure 1).

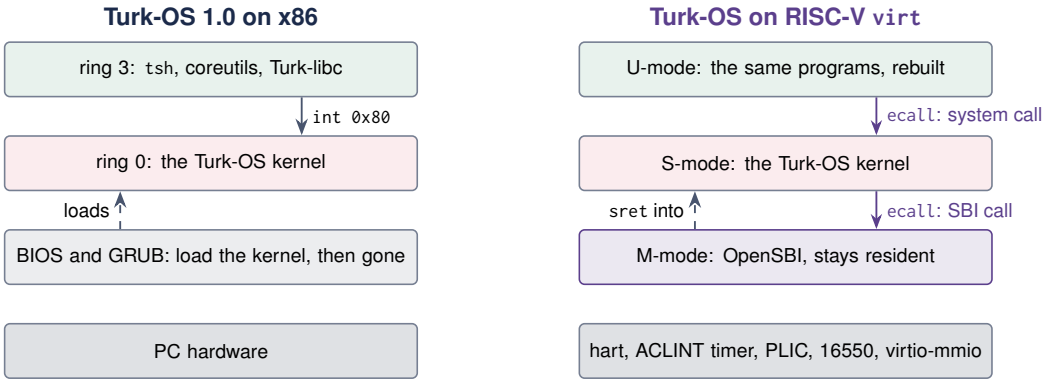


Figure 1: Where the code runs. OpenSBI is to S-mode what the BIOS and GRUB were to ring 0, except that it never leaves.

An SBI call puts the extension ID (EID) in `a7`, the function ID (FID) in `a6` and arguments in `a0`–`a5`, and returns an error code in `a0` and a value in `a1`; other registers are preserved. Turk-OS uses Base (EID `0x10`, to probe), Timer (`0x54494D45`, "TIME"), System Reset (`0x53525354`, "SRST") and Debug Console (`0x4442434E`, "DBCN"), with the legacy putchar (EID `0x01`) as a fallback. OpenSBI v1.7, QEMU 10.2's `-bios default`, lists all four in its banner.

2.3 Control and status registers

What x86 did with special registers and tables, RISC-V does with **control and status registers** (CSRs), read and written by `csrr`, `csrw`, `csrs` (set bits), `csrc` (clear bits) and `csrrw` (swap). The supervisor CSRs of this phase, with their RV32 bit positions from the privileged specification: `sstatus` (bit 1 SIE, interrupts on, like EFLAGS.IF; bit 5 SPIE, SIE before the trap; bit 8 SPP, the mode before the trap, 0 = U; bits 14:13 FS, the FPU state; bit 18 SUM, S-mode may touch U pages); `sie` and `sip` (bit 1 software, 5 timer, 9 external interrupts); `stvec` (the trap handler, like the IDT); `sscratch` (free for the kernel, like the TSS's `esp0`); `sepc`, `scause` and `stval` (the trapping pc, the cause with bit 31 set for interrupts, and the faulting address like CR2); `satp` (bit 31 MODE, 1 = Sv32; bits 30:22 ASID; bits 21:0 the root table's PPN, like CR3); and the read-only 64-bit `time` counter (`rdtime`, `rdtimeh`).

2.4 What happens on a trap

Exceptions, interrupts and system calls all jump to `stvec`. The hardware writes `sepc`, `scause` and `stval`, moves the old mode to SPP and SIE to SPIE, clears SIE and jumps (Figure 2). It switches no stack and saves no register; `entry.S` does that, and `sret` undoes the CSR part. `sepc` points *at* the trapping instruction, which is right for a page fault; after `ecall` and `ebreak` the kernel must advance it, or the program traps forever.

`scause` replaces the x86 vector (Table 1). Note that division by zero does not trap at all, and that a page fault comes without an error code: whether the page was missing or protected, the kernel finds out by looking it up.

2.5 Interrupts: the timer through the SBI, devices through the PLIC

S-mode sees three interrupt sources, each enabled by a bit in `sie` and all of them by `sstatus`.SIE. The **timer** comparator belongs to M-mode on this board, so you arm it with an SBI call; it fires once, and the handler sets the next deadline. The `time` counter runs at the devicetree's `timebase-frequency`,

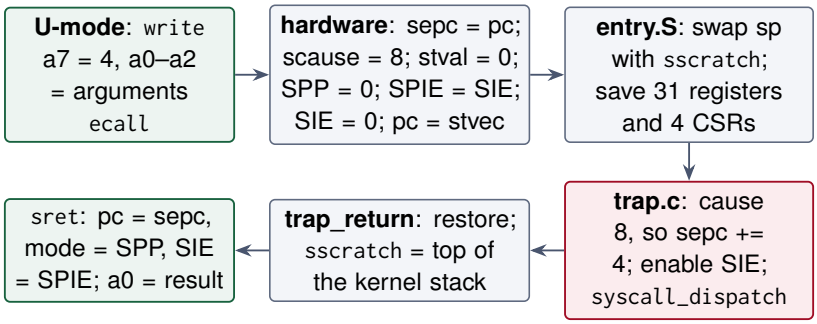


Figure 2: A system call on RISC-V. Compare Figure 1 of Phase 10: no gate, no TSS, no stack switch in hardware.

x86 (Phases 3, 4, 10)	RISC-V scause	Notes
#DE divide error (0)	none	the quotient is all ones
#BP `int3` (3)	3 breakpoint (`ebreak`)	`sepc` points at the `ebreak`
#UD (6), #GP (13)	2 illegal instruction; 5 load, 7 store/AMO access fault	also a privileged CSR from U-mode
#PF (14), CR2	12 instruction, 13 load, 15 store/AMO page fault	address in `stval`
#AC (alignment)	0, 4, 6 address misaligned	OpenSBI emulates loads and stores
`int 0x80`	8 ecall from U-mode	9: from S-mode, OpenSBI's
IRQ 0, the PIT	interrupt 5, supervisor timer	armed through the SBI
IRQ 1, 4, 14	interrupt 9, supervisor external	ask the PLIC which device

Table 1: x86 vectors and RISC-V causes. An interrupt has bit 31 set: a timer interrupt reads `0x80000005`.

10 MHz here. **External** interrupts all arrive as cause 9: the **PLIC** (platform-level interrupt controller) collects the device lines, and the handler reads its *claim* register to learn the source and writes the number back to *complete* it, like the 8259's acknowledge and EOI.

2.6 Sv32: two-level paging, the RISC-V way

Sv32 walks the same tree as x86 (Figure 3): `satp` names a root table of 1024 four-byte entries, VPN[1] selects one, VPN[0] an entry of the second-level table. A root entry can also map a 4 MiB **megapage**. Root entries 768–1023 hold the kernel half, shared by every address space as in Phase 10.

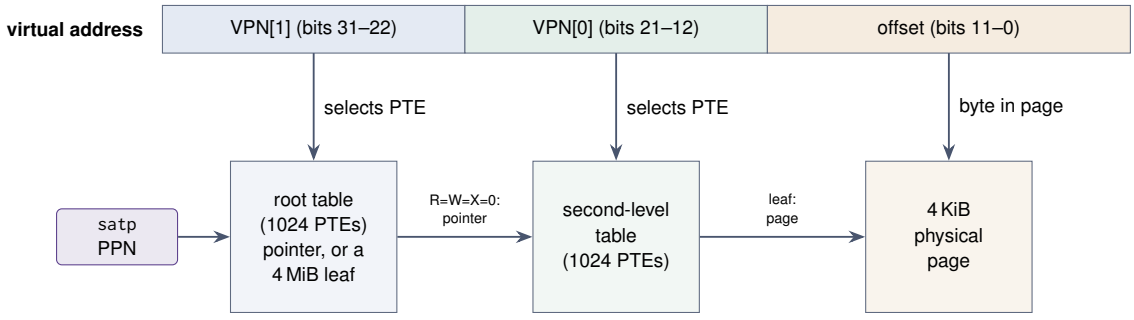


Figure 3: Sv32 translation with 4 KiB pages. Apart from the names, this is Figure 1 of Phase 6.

The entries differ more (Table 2). V (valid) replaces P; an entry with R, W and X all zero points to the next table, any other is a **leaf**; every page has an execute bit. And a core may raise a page fault instead of setting the **A** and **D** bits. QEMU sets them, so the bug hides there, but a simple core, possibly your own, will not. Turk-OS never reads them, so it sets both in every leaf, as the specification recommends.

Question	x86 PDE/PTE (Phase 6)	Sv32 PTE
In use?	P, bit 0	V, bit 0
Table or page?	PS bit in a PDE (4 MiB page)	leaf if any of R, W, X is set
Permissions	R/W bit 1; no execute bit	R, W, X bits 1–3; W needs R
User access	U/S bit 2, in PDE *and* PTE	U bit 4, in the leaf; S-mode needs `sstatus`.SUM to touch it
Accessed, dirty	A bit 5, D bit 6, set by the CPU	A bit 6, D bit 7; may fault instead
Free for the OS	bits 9–11	RSW, bits 8–9 (Turk-OS: bit 8 = COW)
Frame number	bits 31:12, 32-bit physical	bits 31:10, 34-bit physical
Flush one page	`invlpg`	`sfence.vma va, zero`

Table 2: The same questions, answered by two page-table formats. G (global) is bit 8 on x86, bit 5 on Sv32.

2.7 The virt board

QEMU's `virt` board is a small, clean set of devices whose addresses and interrupts OpenSBI passes to the kernel in a **flattened devicetree** (FDT), read by your Task 16.10 parser. Table 3 shows QEMU 10.2 with the default 128 MiB; always take the real values from the FDT.

Address	Size	Device and `compatible` string	PLIC IRQ
`0x00100000`	4 KiB	test device, `sifive,test0`: ends QEMU with a status	–
`0x02000000`	64 KiB	CLINT (ACLINT timer and software interrupts), `riscv,clint0`; M-mode only	–
`0x0C000000`	6 MiB	PLIC, `riscv,plic0`, 95 sources	–
`0x10000000`	256 B	UART, `ns16550a`, 3.6864 MHz clock	10
`0x10001000–0x10008000`	4 KiB each	eight virtio-mmio slots, `virtio,mmio`	1–8
`0x80000000`	128 MiB	RAM; OpenSBI keeps the first 320 KiB	–

Table 3: The RISC-V `virt` board of QEMU 10.2. `/cpus` has `timebase-frequency = 10000000` and `mmu-type = "riscv,sv32"`; `/chosen` has `stdout-path = "/soc/serial@10000000"`.

Run at the physical address first, move to 0xC0000000 in Task 17.7

You could link at `0xC0400000` from day one and enable Sv32 in `boot.S`. Don't, yet: with paging off, each bug in Tasks 17.2–17.6 lives in one new thing (the entry code, a CSR, a device address). Link at the physical address OpenSBI jumps to, keep `satp` at 0, and make `P2V/V2P` the identity by setting `KERNEL_BASE` to `PHYS_BASE` (`0x80000000`) in `<arch/memlayout.h>`. Task 17.7 then changes one number and the linker script and adds a boot page table, and everything above keeps working because it only used the macros: Phases 1–6 of x86, compressed into one phase.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	RISCV	Privileged ISA (Vol. II), chapter *Supervisor-Level ISA*: *Supervisor CSRs*, *Supervisor Instructions*, *Sv32*	–	Every bit, cause and PTE flag of Tasks 17.4–17.9.
REQUIRED	RISCV	SBI specification: ch. 3 *Binary Encoding*, ch. 4 *Base*, ch. 6 *Timer*, ch. 10 *System Reset*, ch. 12 *Debug Console Extension*	–	The firmware calls of Tasks 17.2, 17.5, 17.12.

Priority	Book	What to read	Pages	Why now
RECOMMENDED	RISCV	Unprivileged ISA (Vol. I): *RV32I Base Integer Instruction Set*, *Zicsr*; skim *C* and *A*	–	What you write by hand and read in GDB.
REQUIRED	XV6	Ch. 4 *Traps and system calls*	–	The same trap path on RV64, line by line.
RECOMMENDED	XV6	Ch. 3 *Page tables*; Ch. 5 *Page faults*; Ch. 6 *Interrupts and device drivers*; Ch. 7 *Locking*	–	Sv39 tables, COW, the UART and timer, `amoswap` locks.
REQUIRED	SPEC	Virtio: §2.1 *Device Status Field*, §2.2 *Feature Bits*, §2.7 *Split Virtqueues*; §3.1 *Device Initialization*; §4.2 *Virtio Over MMIO*; §5.2 *Block Device*	–	All of Task 17.10.
RECOMMENDED	SPEC	Devicetree specification §3.4 `/memory`, §3.5 `/reserved-memory`, §3.6 `/chosen`; QEMU's *'virt' Generic Virtual Platform* page for RISC-V	–	What Tasks 17.1 and 17.6 read.
RECOMMENDED	OSTEP	Ch. 20 §20.3 *Multi-level Page Tables* (re-read)	215–222	Sv32 is this design with other bit names.
OPTIONAL	OSTEP	Ch. 6 *Mechanism: Limited Direct Execution* (re-read with Figure 2)	49–62	Which part of a trap is hardware, which is yours.
OPTIONAL	OSTEP	Ch. 28 §28.6–28.11 (test-and-set to fetch-and-add, with LL/SC)	306–313	`amoswap.w` is test-and-set; `lr.w`/`sc.w` are LL/SC.

Keep the *RISC-V ELF psABI* (`https://github.com/riscv-non-isa/riscv-elf-psabi-doc`) at hand for the calling convention and the ELF flags of Task 17.11.

Real-system lens: how xv6-riscv does it

xv6 runs on the same board. It has no firmware: it boots with `-bios none`, `kernel/entry.S` starts in M-mode, and `start.c` delegates all traps to S-mode, opens the PMP, enables Sstc for the timer and `mrets` into `main`; your kernel never touches an M-mode CSR. It is RV64 with Sv39, so `walk()` in `vm.c` descends three levels to your two. Its kernel has its own page table, so `trampoline.S` switches `satp` on every trap; Turk-OS maps the kernel everywhere and needs no trampoline. The rest maps one to one: `kernelvec.S` and `trap.c` are your `entry.S` and `trap.c`, `plic.c` is a dozen lines, `uart.c` drives a 16550, `swtch.S` saves `ra`, `sp` and `s0`–`s11` (in the process structure), `spinlock.c` compiles to `amoswap.w.aq`, and `virtio_disk.c` builds Task 17.10's three-descriptor request.

4 Build it

Task 17.1 Meet the board

Look before you program. Run OpenSBI alone and read its banner: `Domain0 Next Address` is where it will jump (0 without `-kernel`, the ELF entry point with one), `Next Arg1` the FDT it will pass, `Next Mode` S-mode. Then list the memory map and dump the devicetree next to the blobs of Task 16.10.

terminal

```
qemu-system-riscv32 -machine virt -nographic -bios default       # Ctrl-A X quits
qemu-system-riscv32 -machine virt -display none -monitor stdio -S # then: info mtree -f
```

```
qemu-system-riscv32 -machine virt,dumpdtb=tests/data/virt-riscv32.dtb
dtc -I dtb -O dts tests/data/virt-riscv32.dtb | less
```

Find each row of Table 3 in the tree, and note three surprises: `#address-cells` and `#size-cells` are 2 even on this 32-bit machine, so each `reg` number is two cells; a device's PLIC source is its `interrupts` property; and there is no `/reserved-memory`, because OpenSBI adds it to the copy it passes on (Task 17.6). Record it all in `docs/porting.md`.

Verify: Your notes give, without looking again: OpenSBI's next address for an ELF kernel, the FDT address with 128 MiB of RAM (`0x87e00000`, 2 MiB below the end of RAM), the UART's interrupt (10) and the timebase (10 000 000 Hz). The new blob passes your FDT host tests.

Task 17.2 The first instructions

OpenSBI enters in S-mode with `a0` = hart ID, `a1` = the FDT's physical address and `satp` = 0, and promises nothing more. With OpenSBI 1.7 on QEMU 10.2, `sp` still points into OpenSBI's memory, which S-mode may not touch; `stvec` still holds the entry point, so an early trap silently restarts the kernel; and `sstatus`.FS is on, so floating-point registers would leak between processes. `boot.S` (GNU `as` through the C preprocessor, `//` comments) fixes all three, zeroes `.bss` and calls `kmain`.

kernel/arch/riscv32/boot.S

```
    .section .text.boot, "ax"
    .globl _start
_start:                         // S-mode, a0 = hart ID, a1 = FDT (physical), satp = 0
    csrw  sie, zero             // no interrupt sources until arch_init
    li    t0, 0x6000            // sstatus.FS (bits 14:13) = 0: FPU off,
    csrc  sstatus, t0           //   any floating-point instruction traps
    la    t0, early_trap        // stvec still holds OpenSBI's jump address
    csrw  stvec, t0
    la    sp, boot_stack_top    // 16 KiB in .bss, declared further down
    la    t0, _bss_start        // zero .bss (a0 and a1 untouched)
    la    t1, _bss_end
1:  bgeu  t0, t1, store_boot_args
    sw    zero, 0(t0)
    addi  t0, t0, 4
    j     1b
store_boot_args:                // boot_hartid, boot_fdt: two words in .bss,
    la    t0, boot_hartid       //   read later by arch_early_init
    sw    a0, 0(t0)
    la    t0, boot_fdt
    sw    a1, 0(t0)
    call  kmain                 // never returns
    .balign 4                   // stvec needs 4-byte alignment
early_trap:                     // stuck here? read scause and sepc in the monitor
    j     early_trap
```

Link at `0x80400000`, 4 MiB into RAM and clear of OpenSBI (QEMU puts a non-ELF kernel there too). QEMU loads an ELF `-kernel` where its program headers say and passes the entry point to OpenSBI.

kernel/arch/riscv32/kernel.ld (until Task 17.7)

```
OUTPUT_ARCH(riscv)
ENTRY(_start)
SECTIONS {
    . = 0x80400000;  _kernel_start = .;     /* OpenSBI jumps to our entry point */
    .text   : { *(.text.boot) *(.text .text.*) }
    .rodata : ALIGN(4K) { *(.rodata .rodata.* .srodata .srodata.*) }
    .data   : ALIGN(4K) { *(.data .data.* .sdata .sdata.*) }
    .bss    : ALIGN(4K) { _bss_start = .; *(.sbss* .bss .bss.* COMMON) . = ALIGN(16); _bss_end = .; }
    _kernel_end = .;
```

```
}
```

Until the UART works, print through the SBI. `sbi.c` wraps `ecall` once. On top of it, `sbi_putchar(c)` is DBCN function 2 (`console_write_byte`), or the legacy EID `0x01` if Base function 3 (probe) says DBCN is missing, and `sbi_set_timer(t)` is TIME function 0 with the 64-bit deadline split into `a0` (low half) and `a1` (high half), as the RV32 calling convention passes a 64-bit argument. Register the putchar with `console_register()` as the boot console. `arch_early_init` lives here too.

kernel/arch/riscv32/sbi.c

```
struct sbiret { long error; long value; };      /* error in a0, value in a1 */

static struct sbiret sbi_call(long eid, long fid, long arg0, long arg1, long arg2)
{
    register long a0 __asm__("a0") = arg0;
    register long a1 __asm__("a1") = arg1;
    register long a2 __asm__("a2") = arg2;
    register long a6 __asm__("a6") = fid;
    register long a7 __asm__("a7") = eid;
    __asm__ volatile ("ecall" : "+r"(a0), "+r"(a1) : "r"(a2), "r"(a6), "r"(a7) : "memory");
    return (struct sbiret){ a0, a1 };
}
```

kernel/arch/riscv32/arch.mk

```
CROSS       := riscv32-elf-
ARCH_CFLAGS := -march=rv32imac_zicsr_zifencei -mabi=ilp32 -mcmodel=medany
ARCH_SRCS   := $(wildcard kernel/arch/riscv32/*.c kernel/arch/riscv32/*.S)
LDSCRIPT    := kernel/arch/riscv32/kernel.ld
QEMU        := qemu-system-riscv32
QEMU_FLAGS  := -machine virt -bios default -m 128M -nographic -kernel $(BUILD)/kernel.elf
QEMU_TEST_FLAGS  := -append "test" -no-reboot   # the sifive_test exit device is always on virt (Task 17.12)
TEST_PASS_STATUS := 0
```

Verify: `make ARCH=riscv32 run` prints `Merhaba from S-mode: hart 0, FDT at 0x87e00000` below the OpenSBI banner, which now reports `Domain0 Next Address : 0x80400000`; `riscv32-elf-readelf -l` shows one load segment at that address.

Task 17.3 The 16550 again, memory-mapped

The console is the 16550 of Task 2.6, same registers, one byte apart, but memory-mapped; Task 16.8 already split the driver into a core and accessors. Take the address from the FDT: follow `/chosen/stdout-path`, check for `ns16550a` in `compatible`, read `reg` (two cells each, big-endian, or every address comes out as 0) and `interrupts`. Without `reg-shift` the stride is one byte. Initialise it as on x86, register it as the console, and keep the SBI console for panics.

Verify: The boot log looks the same as with the SBI console, and a GDB breakpoint on the MMIO write accessor is hit for every character, writing to `0x10000000`.

Task 17.4 Traps

The RV32 `struct trapframe` holds the 31 registers in order and the four trap CSRs, padded to 144 bytes for 16-byte stack alignment.

kernel/arch/riscv32/include/arch/trapframe.h

```
#define SSTATUS_SIE (1u << 1)
#define SSTATUS_SPIE (1u << 5)
#define SSTATUS_SPP (1u << 8)
#define SSTATUS_SUM (1u << 18)
```

```
struct trapframe {                  /* x1..x31, then the CSRs: 36 words = 144 bytes */
    unsigned long ra, sp, gp, tp, t0, t1, t2, s0, s1, a0, a1, a2, a3, a4, a5, a6, a7;
    unsigned long s2, s3, s4, s5, s6, s7, s8, s9, s10, s11, t3, t4, t5, t6;
    unsigned long sepc, sstatus, scause, stval, pad;
};

static inline unsigned long tf_sysno(const struct trapframe *tf)      { return tf->a7; }
static inline unsigned long tf_arg(const struct trapframe *tf, int n) { return (&tf->a0)[n]; }
static inline void tf_set_ret(struct trapframe *tf, unsigned long v)  { tf->a0 = v; }
static inline uintptr_t tf_pc(const struct trapframe *tf)            { return tf->sepc; }
static inline bool tf_from_user(const struct trapframe *tf)          { return !(tf->sstatus & SSTATUS_SPP); }
    ↪
/* tf_set_pc and tf_sp in the same style */
```

`entry.S` needs a free register before it can save any, and `sp` may be a user's. The convention: `sscratch` holds the top of the task's kernel stack while in U-mode, and 0 in S-mode. Swap `sp` and `sscratch` first; a non-zero result means the trap came from U-mode, zero means swap back and stay on the kernel stack. `trap_return` refills `sscratch` only when going back to U-mode. (`.irp` repeats a line for a list.)

kernel/arch/riscv32/entry.S

```
#define TF_SIZE 144
#define TF_SP 4
#define TF_SEPC 124
#define TF_SSTATUS 128
    .section .text
    .globl trap_entry, trap_return
    .balign 4
trap_entry:                         // stvec points here (direct mode)
    csrrw sp, sscratch, sp          // from U-mode: sp = kernel stack, sscratch = user sp
    bnez  sp, 1f
    csrrw sp, sscratch, sp          // from S-mode (sscratch was 0): undo the swap
1:  addi  sp, sp, -TF_SIZE
    .irp n, 1,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31
    sw    x\n, (\n - 1) * 4(sp)     // every register but sp (x2)
    .endr
    csrrw t0, sscratch, zero        // t0 = user sp, or 0 from S-mode; sscratch = 0 now
    bnez  t0, 2f
    addi  t0, sp, TF_SIZE           // from S-mode: sp before the trap
2:  sw    t0, TF_SP(sp)
    csrr  t1, sepc
    csrr  t2, sstatus
    csrr  t3, scause
    csrr  t4, stval
    sw    t1, TF_SEPC(sp)           // sepc, sstatus, scause, stval: offsets 124-136
    sw    t2, TF_SSTATUS(sp)
    sw    t3, TF_SSTATUS + 4(sp)
    sw    t4, TF_SSTATUS + 8(sp)
    li    t5, 1 << 18               // clear SUM: only uaccess sets it (Task 17.7);
    csrc  sstatus, t5               //   sret restores the saved value
    mv    a0, sp
    call  trap_handler              // void trap_handler(struct trapframe *tf)
trap_return:                        // sp = the frame to resume
    lw    t1, TF_SEPC(sp)
    csrw  sepc, t1
    lw    t1, TF_SSTATUS(sp)
    csrw  sstatus, t1               // saved SIE is 0: no interrupts from here on
    andi  t1, t1, 0x100             // SPP set: back to S-mode, sscratch stays 0
    bnez  t1, 3f
    addi  t1, sp, TF_SIZE           // to U-mode: the next trap from there
    csrw  sscratch, t1              //   starts on this kernel stack, empty again
```

```
3:  .irp n, 1,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31
    lw    x\n, (\n - 1) * 4(sp)
    .endr
    lw    sp, TF_SP(sp)             // last: the user sp, or the interrupted kernel sp
    sret                            // pc = sepc, mode = SPP, SIE = SPIE
```

`arch_init` sets `stvec` to `trap_entry` and `sscratch` to 0. In C, user exceptions become signals; anything unexpected in S-mode gets a crash report with the cause name of Table 1. A breakpoint may be `c.ebreak` (2 bytes) or `ebreak` (4), and the assembler picks, so read the low bits.

kernel/arch/riscv32/trap.c

```c
void trap_handler(struct trapframe *tf)
{
    unsigned long cause = tf->scause;
    if (cause & (1ul << 31)) {                      /* interrupt 5: timer, 9: PLIC    */
        if ((cause & 0xff) == 5) timer_tick(); else plic_dispatch();
        return;
    }
    if (cause == 8) {                               /* ecall from U-mode (Task 17.9)  */
        tf->sepc += 4;                              /* resume after the ecall         */
        csr_set(sstatus, SSTATUS_SIE);              /* system calls can be preempted  */
        syscall_dispatch(tf);
    } else if (cause == 3 && !tf_from_user(tf)) {   /* a kernel breakpoint            */
        uint16_t low = *(volatile uint16_t *)tf->sepc;
        tf->sepc += ((low & 3) == 3) ? 4 : 2;       /* 32-bit encodings end in 11     */
    } else if (cause == 12 || cause == 13 || cause == 15) {
        page_fault(tf);                             /* generic, via arch_fault_info   */
    } else if (tf_from_user(tf)) {
        kill_current(cause == 2 ? SIGILL : cause == 3 ? SIGTRAP : SIGSEGV);
    } else {
        crash_report(tf, cause_name(cause));       /* registers, sepc, stval; panics */
    }
}
```

Verify: `__asm__ volatile ("ebreak")` in `kmain` prints "breakpoint at …" and the kernel carries on; inside `.option norvc` the same test advances `sepc` by 4 instead of 2. Reading `*(volatile int *)0` gives a crash report with "load access fault" and `stval=00000000`: with paging off, address 0 is no memory.

Task 17.5 Time and interrupts

Read `timebase-frequency` from `/cpus`; `arch_timer_init(hz)` computes the interval (10 000 000 / 100 = 100 000 counts), calls `sbi_set_timer(now + interval)` and sets `sie`.STIE (bit 5). On RV32 the 64-bit `time` counter comes in two halves: read `rdtimeh`, `rdtime`, `rdtimeh` again, and retry if the high half changed. `arch_cycles` returns that value (timebase counts, not CPU cycles). `timer_tick` adds the interval to the *previous* deadline, so the tick does not drift, calls `sbi_set_timer` again (without it the timer fires exactly once) and then the generic tick code of Phases 4 and 8.

On QEMU `virt` the PLIC context for hart 0 in S-mode is context 1, as the PLIC node's `interrupts-extended` property says. `plic_init(base)` maps three of the PLIC's pages with `devmap()` (Task 17.7; until then it returns the physical address): the priorities (source i at base $+4i$), the enable bits of context 1 (base `+ 0x2080`), and its threshold and claim registers (base `+ 0x201000` and `+ 0x201004`). It sets the threshold to 0 and `sie`.SEIE (bit 9).

`arch_irq_unmask(irq)` sets the source's priority to 1 (0 means never) and its enable bit; the dispatcher claims, calls, completes:

kernel/arch/riscv32/plic.c

```
void plic_dispatch(void)                        /* scause = interrupt 9          */
{
    uint32_t irq;
    while ((irq = ctx[1]) != 0) {               /* claim (base + 0x201004)       */
        if (irq_handlers[irq])
            irq_handlers[irq](irq);
        ctx[1] = irq;                           /* complete, or it never returns */
    }
}
```

Input comes from the UART now. Set the 16550's interrupt enable register to 1 (data received), register a handler for source 10 and unmask it; the handler in `uart16550.c` passes bytes to `tty_input_char()` while bit 0 of the line status register is set. With `sstatus`.SIE set in `arch_init`, Task 4.6's monitor runs over serial.

Verify: `uptime` in the monitor advances by one second per second. Typed characters arrive by interrupt (a breakpoint in the UART handler is hit), and nothing polls the UART.

Task 17.6 Memory from the devicetree

Fill `struct bootinfo` from the FDT that OpenSBI passes in `a1`, not the dumped one: OpenSBI adds `/reserved-memory` nodes (`no-map`) for its 320 KiB at `0x80000000`. Phase 5's allocator is unchanged.

Usable RAM is the `reg` of each `memory@...` node. Reserved are the children of `/reserved-memory`, the FDT header's reservation block, the FDT itself (`totalsize` bytes) and the kernel image (`_kernel_start`–`_kernel_end`). `/chosen` gives the command line (`bootargs`, from `-append`), the initrd (`linux,initrd-start` and `-end`, from `-initrd`), the console (`stdout-path`) and `rng-seed`, a seed for `arch_entropy`.

To capture what the kernel receives, use the QEMU monitor: `pmemsave 0x87e00000 0x10000 "passed.dtb"`, and add the blob to the FDT tests.

Verify: `mem` reports 128 MiB (256 MiB with `-m 256M`) and lists `0x80000000`–`0x8004ffff`, the kernel and the FDT as reserved. A test that allocates every free frame finds none in a reserved range, and the Phase 5 allocator tests pass.

Task 17.7 Sv32 and the higher half

Now move. In `<arch/memlayout.h>`, set `KERNEL_BASE` to `0xC0000000` and keep `PHYS_BASE` at `0x80000000`, so `P2V(pa)` is `pa - PHYS_BASE + KERNEL_BASE`: RAM appears at `0xC0000000`, the kernel at `0xC0400000`, and the direct map may cover 512 MiB as on x86. In `kernel.ld`, start at `0xC0400000`, give every output section a load address `AT(ADDR(...) - 0x40000000)`, and make the entry point physical (`ENTRY(_start_phys)` with `_start_phys = _start - 0x40000000`), because OpenSBI jumps there with translation off.

`boot.S` still runs at physical addresses, which works because `la` is pc-relative. A static table maps RAM with 4 MiB megapages twice: root entries `0x200`–`0x27F` map `0x80000000`+ to itself (only until the jump) and `0x300`–`0x37F` map `0xC0000000`+ to `0x80000000`+, 512 MiB each. GNU `as` builds such a table at assembly time with `.rept 128`, `.word ((pa >> 12) << 10) | 0xCF` (V, R, W, X, A, D) and `.set pa, pa + 0x400000`; `.zero` fills the gaps. Then:

kernel/arch/riscv32/boot.S (paging part)

```
    .section .text.boot, "ax"
enable_paging:                          // (first: boot_pgdir[0x3FF] -> dev_pt, see below)
    la    t1, boot_pgdir                // physical, since la is pc-relative
    srli  t1, t1, 12
    li    t2, 1 << 31                   // satp.MODE = Sv32
```

```
    or    t1, t1, t2
    csrw  satp, t1                    // translation on; the identity entries
    sfence.vma                        //   keep the next fetch working
    lui   t0, %hi(higher_half)        // the absolute link address 0xC04...
    addi  t0, t0, %lo(higher_half)
    jr    t0
higher_half:                          // next: stvec = trap_entry (it held a physical
    la    sp, boot_stack_top          //   address), zero the 128 identity entries,
    j     store_boot_args             //   sfence.vma; then Task 17.2's code and kmain
```

Devices live in the last 4 MiB: root entry `0x3FF` points to a page table `dev_pt` in `.bss`, and `devmap(pa)` in `sv32.c` fills its next entry with a read-write leaf (A and D set) and returns the address. On RISC-V, whether an address is RAM or a device is fixed by the platform, so Sv32 has no cache bits like x86's PCD.

The rest of `sv32.c` is Phase 6's VMM in RISC-V notation, with `PTE_PPN(pa)` = `((pa >> 12) << 10)` and `PTE_COW` = RSW bit 8:

kernel/arch/riscv32/sv32.c

```
static pte_t vm_to_pte(unsigned flags)
{
    pte_t p = PTE_V | PTE_A;                  /* A and D set by us: no fault to set them */
    if (flags & VM_READ)  p |= PTE_R;
    if (flags & VM_WRITE) p |= PTE_R | PTE_W | PTE_D;     /* W without R is reserved */
    if (flags & VM_EXEC)  p |= PTE_X;
    if (flags & VM_USER)  p |= PTE_U;
    if (flags & VM_COW)   p = (p & ~PTE_W) | PTE_COW;     /* read-only until written */
    return p;
}

static pte_t *walk(pte_t *root, uintptr_t va, bool create)
{
    pte_t *pde = &root[va >> 22];                         /* VPN[1] */
    if (!(*pde & PTE_V)) {
        paddr_t pt = create ? pmm_alloc_frame() : 0;
        if (!pt)
            return NULL;
        memset(P2V(pt), 0, PAGE_SIZE);
        *pde = PTE_PPN(pt) | PTE_V;                       /* R = W = X = 0: a pointer */
    }
    ASSERT(!(*pde & (PTE_R | PTE_W | PTE_X)));            /* not inside a megapage    */
    pte_t *table = P2V(PTE_ADDR(*pde));
    return &table[(va >> 12) & 0x3FF];                    /* VPN[0] */
}
```

`arch_mmu_switch` writes `(1 << 31) | (V2P(root) >> 12)` to `satp`, then `sfence.vma zero, zero`; `arch_tlb_flush(va)` is `sfence.vma va, zero`; map, unmap, lookup and protect use `walk`. `arch_mmu_init_space` copies root entries 768–1023 from `boot_pgdir`, now the kernel's address space, so pre-allocate the kernel tables as in Task 6.6. `arch_fault_info` takes `addr` from `stval`, `write` from cause 15, `exec` from 12, `user` from SPP, and `present` from a lookup. Task 11.4's copy-on-write then works unchanged.

With `sstatus`.SUM clear, any S-mode access to a U page faults, `copy_from_user` included. That is the protection you want everywhere except inside uaccess, the RISC-V counterpart of x86's SMAP. So SUM is set only by `arch_uaccess_begin()` (`csrs sstatus` with bit 18) and cleared by `arch_uaccess_end()` (`csrc`); `entry.S` clears it on every trap, and `sret` restores the saved value, so a timer interrupt in the middle of a copy neither leaks SUM into another task nor loses it. `arch_signal_frame` and `arch_sigreturn` reach the user stack through `copy_to_user` and `copy_from_user` like every other caller. A stray dereference of a user pointer anywhere else in the kernel is now a load or store page fault in S-mode, and the crash report names it.

`copy_to_user` still asks `fault_in_user_page` for write access to every page before it copies, so `ro_read` gets `-EFAULT` and `cow_read` gets its private copy before the first store. If a store page fault (cause 15) does arrive from inside uaccess, `page_fault` resolves it through the same fault-in function; a store fault on a user address from anywhere else in S-mode is a kernel bug and panics.

Sv32 has an execute bit, so the RISC-V kernel can have what 32-bit x86 paging can't: no page both writable and executable. The boot table maps all of RAM read-write-execute, which is fine for the jump and no longer. When `kmain` builds the kernel's address space, replace the megapage at `0xC0400000` with a table of 4 KiB leaves, `.text` R and X, `.rodata` R, `.data` and `.bss` R and W (symbols from `kernel.ld` mark the edges), and give every other RAM megapage R and W only. Pre-allocate that table with the others, so `arch_mmu_init_space` shares it. User pages follow `vm_to_pte`: only an ELF segment with `PF_X` gets `VM_EXEC`, so the stack, the heap and data are never executable.

Verify: `info registers` shows the pc at `0xc04...`; the monitor's `info mem` lists the kernel text without `w`, the rest of RAM from `c0000000` without `x`, the device pages at `ffc00000` and nothing below `0xC0000000`. Reading address 0 now gives a "load page fault", and the VMM tests of Task 6.7 pass. A write to a kernel function faults ("store page fault", `stval` in `.text`), and so does a kernel read of a user pointer outside uaccess.

Task 17.8 Tasks and locks

`arch_switch` is Phase 8's `switch_context`: save `ra` and `s0`–`s11` on the old stack, swap stack pointers, restore.

kernel/arch/riscv32/switch.S

```
// void arch_switch(struct context **save, struct context *load)
    .section .text
    .globl arch_switch, task_trampoline
arch_switch:
    addi  sp, sp, -64               // 13 words, rounded up for 16-byte alignment
    .set  off, 0
    .irp  r, ra, s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11
    sw    \r, off(sp)
    .set  off, off + 4
    .endr
    sw    sp, 0(a0)                 // *save = sp
    mv    sp, a1                    // the new task's kernel stack
    .set  off, 0
    .irp  r, ra, s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11
    lw    \r, off(sp)
    .set  off, off + 4
    .endr
    addi  sp, sp, 64
    ret                             // to where the new task called arch_switch
task_trampoline:                    // ra of a new task (arch_task_init)
    csrsi sstatus, 2                // SIE = 1: Phase 8's sti
    mv    a0, s1                    // entry(arg), with s0 = entry, s1 = arg
    jalr  s0
    call  task_exit                 // never returns
```

`arch_task_init` fakes a 64-byte context (`ra` = `task_trampoline`, `s0` = entry, `s1` = argument) as in Task 8.4. `irq_save` is `csrrci` on SIE, which clears it and returns the old `sstatus`; `cpu_idle` is `wfi`, which wakes for an enabled pending interrupt even while SIE is 0.

kernel/arch/riscv32/include/arch/atomic.h

```
#ifdef __riscv_atomic                   /* defined by the compiler when 'a' is in -march */
#define ARCH_HAS_ATOMICS 1
static inline uint32_t atomic_xchg(volatile uint32_t *p, uint32_t v)
{
```

```
    uint32_t old;
    __asm__ volatile ("amoswap.w.aq %0, %2, %1" : "=r"(old), "+A"(*p) : "r"(v) : "memory");
    return old;
}
#else
#define ARCH_HAS_ATOMICS 0              /* one hart, no A extension (Task 17.13) */
#endif
```

`amoswap.w.aq` is Task 9.2's test-and-set with acquire ordering; release with `__atomic_store_n` and `__ATOMIC_RELEASE`. (`lr.w`/`sc.w` serve more complex updates.) With `ARCH_HAS_ATOMICS` 0 a spinlock is only `irq_save`, which on one hart is mutual exclusion; there, avoid `__atomic` builtins, which become calls such as `__atomic_exchange_4` that no kernel has.

Verify: Phase 8's demo threads preempt each other, the Phase 9 tests (bounded buffer, dining philosophers) pass, and 1000 create/exit cycles leave the frame count unchanged.

Task 17.9 Into user mode and back

`arch_user_entry` prepares a frame for `sret`; `arch_return_to_user` (`user.S`) clears SIE, sets `sp` to the frame at the top of the kernel stack and jumps to `trap_return`.

kernel/arch/riscv32/trap.c (user entry)

```
void arch_user_entry(struct trapframe *tf, uintptr_t entry, uintptr_t sp)
{
    memset(tf, 0, sizeof *tf);              /* no kernel values in user registers */
    tf->sepc = entry;                       /* sret jumps here                    */
    tf->sp = sp;                            /* argc, argv, envp (Task 11.6)       */
    unsigned long s = csr_read(sstatus);
    s &= ~(SSTATUS_SPP | SSTATUS_SIE | SSTATUS_SUM | SSTATUS_FS);  /* SPP = 0: U */
    tf->sstatus = s | SSTATUS_SPIE;         /* interrupts on after sret           */
}
```

A system call is cause 8; `sepc += 4` comes *before* `syscall_dispatch`, because `execve` and `sigreturn` replace `sepc`, and `fork` copies the frame. Arguments come from `a0`–`a5`, the number from `a7`: still the Linux i386 number (`write` is 4, not Linux RISC-V's 64). Task 10.9's evil programs give two new answers. `divzero` is *not* killed (its quotient is -1; print it), and `cli` becomes `csrci sstatus, 2`, killed as an illegal instruction (2). `nullptr` and `kpeek` die of a load page fault (13); `badptr`, `hugelen` and `badsys` get `-EFAULT`, `-EFAULT` and `-ENOSYS` as on x86. The generic ones of Task 16.7, `ro_read`, `lazy_read` and `cow_read` among them, must give exactly the x86 answers, which proves that `SUM` and the shared fault-in path work. One more is new because RISC-V can enforce it: `nxdata` copies a `ret` instruction into a heap buffer and calls it, and dies of an instruction page fault (12), since the heap has no X bit.

Verify: An RV32 `hello` prints "Merhaba from U-mode, pid 3" and exits, a looping program is preempted, and every evil program behaves as in the table in one boot: `ro_read` returns `-EFAULT` and the kernel carries on, `cow_read`'s data is seen only by the reader, and `nxdata` is killed with cause 12.

Task 17.10 A disk on virtio

The disk is a virtio block device (Concepts aside: a virtio device shares rings of buffer descriptions with the driver instead of imitating old hardware). QEMU's virtio-mmio defaults to the legacy interface (version 1); ask for version 2:

kernel/arch/riscv32/arch.mk (QEMU_FLAGS, continued)

```
-global virtio-mmio.force-legacy=false \
-drive file=$(BUILD)/turkfs.img,if=none,format=raw,id=hd0 \
-device virtio-blk-device,drive=hd0,bus=virtio-mmio-bus.0       # slot 0x10001000, IRQ 1
```

Keep `virtio_mmio.c` and `virtio_blk.c` free of RISC-V, since Phase 18 reuses them. The arch code offers each `virtio,mmio` node, mapped with `devmap`; empty slots have device ID 0. The `VIRTIO_MMIO_*` offsets come from the specification's *Virtio Over MMIO*: magic `0x000`, version `0x004`, device ID `0x008`, features `0x010`–`0x024`, queue registers `0x030`–`0x044`, notify `0x050`, interrupt status and ACK `0x060`/`0x064`, status `0x070`, queue addresses `0x080`/`0x090`/`0x0a0`, configuration from `0x100`.

kernel/drivers/virtio_mmio.c (probe and handshake)

```
#define R(off) regs[(off) / 4]

int virtio_mmio_setup(volatile uint32_t *regs, uint32_t device_id, struct virtq *q)
{
    if (R(VIRTIO_MMIO_MAGIC) != 0x74726976 || R(VIRTIO_MMIO_DEVICE_ID) != device_id)
        return -ENODEV;                         /* not "virt", or another device */
    if (R(VIRTIO_MMIO_VERSION) != 2)
        return -ENODEV;                         /* legacy: see force-legacy above */
    R(VIRTIO_MMIO_STATUS) = 0;                  /* reset */
    R(VIRTIO_MMIO_STATUS) = ST_ACKNOWLEDGE;
    R(VIRTIO_MMIO_STATUS) |= ST_DRIVER;
    R(VIRTIO_MMIO_DEV_FEAT_SEL) = 1;            /* feature bits 32-63 */
    if (!(R(VIRTIO_MMIO_DEV_FEAT) & 1))         /* bit 32: VIRTIO_F_VERSION_1 */
        return -ENODEV;
    R(VIRTIO_MMIO_DRV_FEAT_SEL) = 1;  R(VIRTIO_MMIO_DRV_FEAT) = 1;   /* accept only it */
    R(VIRTIO_MMIO_DRV_FEAT_SEL) = 0;  R(VIRTIO_MMIO_DRV_FEAT) = 0;
    R(VIRTIO_MMIO_STATUS) |= ST_FEATURES_OK;
    if (!(R(VIRTIO_MMIO_STATUS) & ST_FEATURES_OK))
        return -ENODEV;                         /* the device refused */
    R(VIRTIO_MMIO_QUEUE_SEL) = 0;               /* queue 0; check QueueNumMax first */
    R(VIRTIO_MMIO_QUEUE_NUM) = QSIZE;           /* e.g. 8 */
    set_addr(regs, VIRTIO_MMIO_QUEUE_DESC_LOW, q->desc);     /* physical: low, high */
    set_addr(regs, VIRTIO_MMIO_QUEUE_DRIVER_LOW, q->avail);
    set_addr(regs, VIRTIO_MMIO_QUEUE_DEVICE_LOW, q->used);
    R(VIRTIO_MMIO_QUEUE_READY) = 1;
    R(VIRTIO_MMIO_STATUS) |= ST_DRIVER_OK;      /* live from here on */
    return 0;
}
```

The split virtqueue has three parts, a page each from `kalloc_page`: the descriptor table (buffer address, length, flags NEXT and WRITE, next index), the available ring (requests from the driver) and the used ring (completions from the device). A request chains three descriptors: the header `struct virtio_blk_req { uint32_t type, reserved; uint64_t sector; }` (type 0 read, 1 write; NEXT), the buffer (NEXT, plus WRITE for a read) and a status byte (WRITE; 0 = OK). Post the head in the available ring, fence (`__sync_synchronize()`), bump its `idx`, fence, write 0 to `QueueNotify`, and sleep. The handler (PLIC source 1) acknowledges `InterruptStatus`, walks the used ring and wakes the waiter. Sectors are 512 bytes. Register `vda` through Task 12.5's `struct blockdev`, and the buffer cache and TurkFS sit on top unchanged.

The used ring is written by the device, so treat its `id` like any other outside input: check it against `QSIZE` before using it as an index into your request table, and log and skip an entry that is out of range. A virtio device can DMA anywhere in guest memory, so this is no defence against a hostile one (QEMU's devices are trusted, as the threat model of Task 16.10 says), but the check costs one comparison and turns a device bug into a log line instead of a write through a wild pointer.

kernel/drivers/virtio_blk.c (interrupt handler)

```
while (q->last_used != q->used->idx) {
    __sync_synchronize();                       /* read idx before the entry */
    uint32_t id = q->used->ring[q->last_used % QSIZE].id;
    q->last_used++;
    if (id >= QSIZE) {                          /* from the device: never trust it */
```

```
        pr_err("virtio: used id %u out of range\n", (unsigned)id);
        continue;
    }
    q->req[id].done = true;                     /* the request that used head id */
    wake_up_one(&q->req[id].wq);
}
```

Physical addresses for every buffer

`V2P` is only right for direct-map addresses. A 1 KiB buffer from the Phase 7 heap can straddle two pages that are not adjacent in RAM, and the device writes half of it over someone else's memory. Take buffer data from `kalloc_page` (four per page), or split such a buffer into two descriptors.

Verify: The kernel prints `vda: virtio-blk, 131072 sectors (64 MiB)` (from the `capacity` field at `0x100`); the Phase 12 tests pass on `vda`; a pattern written to sector 100 shows on the host with `hexdump -C -s $((100*512)) -n 64 build/riscv32/disk.img`.

Task 17.11 Userland for RV32

Turk-libc and the utilities do not change. The system call stub sets `a7` and `a0`–`a5`, executes `ecall` and returns `a0`, with register variables like `sbi_call`; `user.ld` still starts at `0x00400000`; and `crt0.S` reads the System V stack of Task 11.6:

user/arch/riscv32/crt0.S

```
    .section .text
    .globl _start
_start:                             // sp -> argc, argv[], NULL, envp[], NULL
    lw    a0, 0(sp)                 // argc
    addi  a1, sp, 4                 // argv
    slli  t0, a0, 2
    add   a2, a1, t0
    addi  a2, a2, 4                 // envp = argv + argc + 1
    andi  sp, sp, -16               // calls need a 16-byte aligned sp
    call  main                      // main(argc, argv, envp)
    call  exit                      // exit(main's result), already in a0
```

Build with the kernel's `-march`/`-mabi`, `-nostdlib -static` and `-lgcc` (RV32 has no 64-bit divide; GCC calls `__udivdi3`). The ELF loader expects `EM_RISCV` (243), `ELFCLASS32`, little-endian, and `e_flags & 0x0006` = 0 (soft float: the kernel saves no FPU registers); bit `0x0001` (compressed) is fine. Rebuild the root archive (Task 13.4) and the TurkFS image, and pass the archive with `-initrd`.

Verify: `make ARCH=riscv32 run` boots to `turk@os:/$`; `ls /bin`, `echo merhaba | cat`, a redirection and Ctrl+C on `cat` behave as on x86; an i386 ELF fails `execve` with `-ENOEXEC`.

Task 17.12 make test on RISC-V

The test device replaces `isa-debug-exit`: a 32-bit write of `0x5555` ends QEMU with status 0, and `(code << 16) | 0x3333` with status `code` (`0x7777` resets). So a pass is 0 here, not 1: set `TEST_PASS_STATUS := 0` in `arch.mk`, and fail with a non-zero code, since code 0 also exits with 0. `arch_halt` is SRST function 0 with reset type 0 (shutdown). `-append "test"` reaches `/chosen/bootargs`, so the test target stays generic. Add `riscv32` to the CI matrix of Task 16.11, with the evil programs and a short run of the fuzzer of Phase 14. Fuzz for an hour locally as well (new code sits under the system call boundary), then run Task 15.5's benchmarks and put them beside x86 in `docs/porting.md`. User-space `rdtime` needs `scounteren`.TM (bit 1); one count is 100 ns. Both machines are emulated on one host, so compare ratios, not absolute times.

Verify: `make ARCH=riscv32 test` prints `ALL TESTS PASSED`, a deliberately broken test makes it fail with status 1, `make ARCH=i386 test` still passes, the CI matrix is green for `riscv32`, and an hour of fuzzing ends without a panic.

Task 17.13 Optional: Turk-OS on Timur-RV32IMC

This depends on your hardware project: start when Timur passes its own tests and runs a bare-metal program on its UART. Check what the core offers, fill the gaps in firmware, and build a kernel that needs nothing more.

Requirement	Why Turk-OS needs it	Workaround if missing
Zicsr; M, S, U modes; Sv32 with `sfence.vma`	traps; protection; address spaces, COW, the higher half	none
Precise traps, `medeleg/mideleg`	faults restart the instruction; S-mode handles them	an M-mode handler forwarding to `stvec`
`mtime`, `mtimecmp`	the timer behind `sbi_set_timer`	none: the tick is essential
A 16550-like UART, an interrupt controller	console; input by interrupt	another driver; poll the UART on each tick
A extension	`amoswap` spinlocks	build `rv32imc`, `ARCH_HAS_ATOMICS=0`
Misaligned access	compiled code may assume it	`-mstrict-align` or M-mode emulation
A boot loader and a devicetree	kernel, initrd and FDT in RAM; all boot facts	UART or JTAG loading; a hand-written `timur.dts` for `dtc`

Something must run in M-mode. Either port OpenSBI, whose *generic* platform is driven by the FDT, or write a small SBI shim: an `mtvec` handler for Base probes, TIME (program `mtimecmp`; on the machine timer interrupt set `mip`.STIP and mask MTIE), DBCN and SRST, with everything else delegated through `medeleg` and `mideleg`, ending in an `mret` to the kernel with `a0` = hart ID and `a1` = FDT. Develop it on QEMU: `-bios build/shim.elf -kernel build/riscv32/kernel.elf` runs your shim in M-mode at `0x80000000` instead of OpenSBI (`-bios none` boots one image in M-mode, as xv6 does). Build the Timur kernel with `-march=rv32imc_zicsr_zifencei`, an initrd-only root and the UART console, and fill the table with Timur's real answers in `docs/porting.md`.

Verify: Your shim boots the unchanged QEMU kernel to the shell, and the `rv32imc` kernel contains no `amoswap`, `lr.w` or `sc.w` (`riscv32-elf-objdump -d`). On Timur, the milestone is the banner, then the prompt.

5 Testing and debugging

`-d int,guest_errors -D build/riscv32/qemu.log` logs every trap with its cause, `sepc` and `stval`. With `-s -S`, a GDB that knows RISC-V (Task 16.1) and `set architecture riscv:rv32` steps through `boot.S`. The monitor's `info registers` shows every CSR and `info mem` the live Sv32 mappings.

Symptom	Likely cause	What to do
`unrecognized opcode 'csrr a0,sstatus', extension 'zicsr' required`	Current binutils keep the CSR instructions out of the base ISA	Add `_zicsr_zifencei` to `-march`.
OpenSBI's banner, then nothing	Linked at the wrong address, or a virtual entry point	`readelf -l`; the banner's `Domain0 Next Address` must be your entry.
The banner repeats forever	A trap before `stvec` was set, or a call on OpenSBI's `sp`	Set `stvec` and `sp` first in `boot.S`.
`ebreak` or `ecall` loops, or an instruction is skipped	`sepc` not advanced, or by 4 over a 2-byte `c.ebreak`	`ecall`: +4 before dispatching; `ebreak`: low bits `11` mean 4 bytes.

Symptom	Likely cause	What to do
Load page fault in `copy_from_user`	`sstatus`.SUM clear	Set SUM in `arch_uaccess_begin`, and make sure the copy runs between the two hooks.
The kernel faults at `.text` or `.data` right after `kmain` replaces the boot megapage	A section edge from `kernel.ld` not page-aligned, or the new table installed before it was filled	`. = ALIGN(4096)` before each section; fill the table, then write the root entry and `sfence.vma`.
Fault right after the `satp` write	The pc is not mapped in the new table, or no `sfence.vma`	Keep the identity megapages until the jump.
Every first access to a page faults, on a real core	A or D clear, and the core faults instead of setting them	Set A always, D on writable leaves.
PLIC interrupts never arrive	Priority 0, the wrong context, SEIE clear, or a claim never completed	Context 1; priority 1, threshold 0; always complete.
The virtio probe sees version 1	The legacy interface	`-global virtio-mmio.force-legacy=false`.
Corruption near `0x80000000`	OpenSBI's memory given out as free	Reserve `/reserved-memory` and the FDT.

6 Milestone

Milestone: Merhaba, RISC-V

- ☐ `qemu-system-riscv32 -machine virt` boots Turk-OS on OpenSBI to the `tsh` prompt; input arrives by UART interrupt.
- ☐ The kernel runs at `0xC0400000` under Sv32, and every device comes from the devicetree.
- ☐ Every exception gives a readable crash report, `ebreak` returns, and the evil programs are contained.
- ☐ `sstatus`.SUM is set only inside uaccess; no kernel or user page is both writable and executable, and a write to kernel text and an execute from a user data page both fault; the virtio used-ring `id` is checked.
- ☐ `make ARCH=riscv32 test` passes, `make ARCH=i386 test` still passes, the evil programs and the fuzzer pass on RV32 in CI, and an hour of fuzzing ends without a panic.
- ☐ A TurkFS image written by Turk-OS on RISC-V passes `fsck.minix -f` on the host.
- ☐ `docs/porting.md` records the board, every leak you fixed, and the benchmarks beside x86.
- ☐ (Optional) The Timur-RV32IMC table is filled in, and your SBI shim boots the kernel in QEMU.

Self-check questions

1. What can M-mode do that S-mode cannot? Which jobs of the BIOS and GRUB does OpenSBI take over?
2. What does the hardware do on a trap, and what must `entry.S` do because the hardware doesn't?
3. Why does `ecall` need `sepc += 4` while a page fault must leave `sepc` alone? And `ebreak`?
4. How do an x86 PTE and an Sv32 PTE tell a table from a page, and "absent" from "present but forbidden"?
5. What is `sstatus`.SUM for? Why does Turk-OS set A and D itself when QEMU would?
6. Why can an RV32IMC core without the A extension still run Turk-OS on one hart, and what breaks with two?

7 Going further

Port to **RV64** with Sv39, like xv6. Start more harts with the SBI **HSM** extension and make the scheduler SMP-safe. Use **Sstc** (`stimecmp`, which QEMU's hart has) to drop the SBI call from every tick. Add global kernel mappings and ASIDs. Or run Turk-OS on a real RV64 board, whose boot chain (U-Boot SPL, OpenSBI, U-Boot) is the one QEMU's `virt` imitates.

Next: **Phase 18 The ARM Port** →

Books used in this plan

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. **LOB** and **OSTEP** are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.

Port references used in Stage V

XV6 Russ Cox, Frans Kaashoek, Robert Morris. *xv6: a simple, Unix-like teaching operating system*, RISC-V edition. The companion book to MIT's xv6-riscv kernel. Free at https://pdos.csail.mit.edu/6.1810/; source at https://github.com/mit-pdos/xv6-riscv.

RISCV *The RISC-V Instruction Set Manual*, Volume I (Unprivileged ISA) and Volume II (Privileged Architecture); *RISC-V Supervisor Binary Interface Specification*. Free at https://riscv.org/specifications/ratified/ and https://github.com/riscv-non-isa/riscv-sbi-doc.

ARM *Arm Architecture Reference Manual for A-profile architecture* (DDI 0487), and Arm's *Learn the architecture* guides: AArch64 exception model, AArch64 memory management, Generic Timer, GIC. Free at https://developer.arm.com/documentation.

SPEC *Devicetree Specification* (https://www.devicetree.org/specifications/); *Virtual I/O Device (VIRTIO)*, OASIS (https://docs.oasis-open.org/virtio/virtio/); the QEMU documentation for the RISC-V and Arm `virt` boards (https://www.qemu.org/docs/master/system/).

All four are free. Specifications are revised often, so the reading plans cite them by chapter and section title rather than by page.